Cloud-Edge Collaborative Inference with Large and Small Language Models: A Survey

BEN TROVATO* and G.K.M. TOBIN*, Institute for Clarity in Documentation, USA
LARS THØRVÄLD, The Thørväld Group, Iceland
VALERIE BÉRANGER, Inria Paris-Rocquencourt, France
APARNA PATEL, Rajiv Gandhi University, India
HUIFEN CHAN, Tsinghua University, China
CHARLES PALMER, Palmer Research Laboratories, USA
JOHN SMITH, The Thørväld Group, Iceland
JULIUS P. KUMQUAT, The Kumquat Consortium, USA

A clear and well-documented LaTeX document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the "acmart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

CCS Concepts: • **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Additional Key Words and Phrases: Do, Not, Us, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Ben Trovato, G.K.M. Tobin, Lars Thørväld, Valerie Béranger, Aparna Patel, Huifen Chan, Charles Palmer, John Smith, and Julius P. Kumquat. 2018. Cloud-Edge Collaborative Inference with Large and Small Language Models: A Survey. *Proc. ACM Meas. Anal. Comput. Syst.* 37, 4, Article 111 (August 2018), 19 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The emergence of large language models (LLMs) has undoubtedly brought a significant transformation to the field of natural language processing (NLP). At their core, LLMs aim to capture the probability of generating word sequences, thereby facilitating the prediction of probabilities for upcoming or absent tokens. The ongoing pursuit of enabling machines to truly comprehend and decipher the nuanced meanings of natural language continues to face persistent challenges. Notably, the widespread adoption of the Transformer architecture has

*Both authors contributed equally to this research.

Authors' Contact Information: Ben Trovato, trovato@corporation.com; G.K.M. Tobin, webmaster@marysville-ohio.com, Institute for Clarity in Documentation, Dublin, Ohio, USA; Lars Thørväld, The Thørväld Group, Hekla, Iceland, larst@affiliation.org; Valerie Béranger, Inria Paris-Rocquencourt, Rocquencourt, France; Aparna Patel, Rajiv Gandhi University, Doimukh, Arunachal Pradesh, India; Huifen Chan, Tsinghua University, Haidian Qu, Beijing Shi, China; Charles Palmer, Palmer Research Laboratories, San Antonio, Texas, USA, cpalmer@prl.com; John Smith, The Thørväld Group, Hekla, Iceland, jsmith@affiliation.org; Julius P. Kumquat, The Kumquat Consortium, New York, USA, jpkumquat@consortium.net.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2476-1249/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

revolutionized the training and inference processes of these models by leveraging large-scale unsupervised pre-training. Trained on extensive text corpora, LLMs acquire profound linguistic knowledge and contextual understanding. Prominent examples, such as OpenAI's GPT series, Meta's LLaMA series, and Baidu's Ernie series, have demonstrated unprecedented capabilities in understanding and generating human-like text. From automated customer support and advanced content creation to intelligent education and conversational systems, LLMs have shown immense potential across diverse applications. However, traditional LLMs are predominantly deployed on cloud servers, where their training and inference demand substantial storage, computational resources, communication bandwidth, cost, and security considerations. These requirements have spurred interest in deploying personalized LLMs on edge devices, extending their reach to mobile systems, smart automotive systems, and personal wearable devices. Mobile users are no longer satisfied with single AI inference services; instead, they require multiple inference services with chain dependencies to process data effectively. This paradigm not only addresses various personalized user needs but also reduces the bandwidth and resource costs associated with LLM deployment and cloud-based communication.

The rapid growth of the personalized LLM application market reflects the increasing interest in deploying LLMs at the edge. This growth spans various industries, including manufacturing, education, healthcare, and government sectors. The combination of pre-training and fine-tuning capabilities in LLMs enables them to efficiently acquire domain-specific knowledge, addressing the unique needs of different vertical industries in a more efficient, private, and secure manner. This market trend, alongside technological advancements, underscores the necessity and urgency of deploying LLMs on edge devices. Despite the clear advantages, there are still significant challenges in deploying LLMs at the edge. Edge devices often lack the computational and storage resources required to integrate large, resource-intensive language models. While cloud-based deployments offer various benefits, such as greater processing power, they also come with inherent limitations. Take autonomous vehicles as an example: they rely on querying cloud-based LLMs to make real-time decisions. In areas with low or unstable network connectivity, communication with the cloud may experience delays, potentially causing the vehicle to lag in responding to unexpected road hazards while waiting for crucial information from the LLM. Currently, the main challenges of edge-deployed LLMs include limited computational power, restricted storage, high communication latency, and energy constraints, all of which complicate cloud-based LLM architectures. While deploying lightweight LLMs on edge devices may meet certain low-complexity, simple needs, it is insufficient to fully address the diverse technological and architectural demands of edge computing. As a result, the concept of cloud-edge collaborative deployment of smaller models has emerged as an attractive alternative.

This survey provides a comprehensive examination of edge–cloud collaborative AI systems, with a particular focus on the synergy between large and small language models (LLMs and SLMs). The paper begins by introducing the background, motivation, and key challenges of edge–cloud collaboration for LLMs, while also outlining the scope and structure of the survey. It then establishes the foundational concepts of edge–cloud architectures, detailing the constraints of edge devices, the evolving landscape of language models, and collaborative learning and inference paradigms. Building on this foundation, the survey explores dynamic task allocation and scheduling strategies, including techniques based on request-awareness, value prioritization, and semantic similarity. Subsequent sections delve into speculative decoding and collaborative verification, highlighting draft–refine frameworks, skeleton completion, and token-level validation. The paper also investigates multi-model collaborative architectures, such as hierarchical model cooperation, functional complementarity between generator and verifier models, and pipeline-based inference. To address practical deployment challenges, the survey reviews lightweight adaptation and knowledge transfer approaches, including parameter-efficient tuning, adapter-based modularization, and edge-oriented quantization strategies. Privacy and security issues are also considered, encompassing secure computation, differential privacy, and inference isolation techniques. Furthermore, the survey presents a detailed analysis of current benchmarks, datasets, and evaluation protocols, emphasizing the trade-offs between latency, accuracy, cost, and privacy. Finally, it discusses open challenges

and future directions, including system scalability, adaptive scheduling, robustness, and energy-efficient design. Through this structure, the paper aims to offer a holistic and forward-looking perspective on enabling intelligent, efficient, and secure collaboration between large and small models in edge–cloud environments.

1.1 Background and Motivation

Large Language Models (LLMs) have demonstrated unprecedented capabilities across various natural language understanding and generation tasks. However, the deployment of such models in real-world, resource-constrained environments, such as edge devices, poses significant challenges due to their computational and memory demands. To bridge the gap between the high-performance requirements of LLMs and the limited resources of edge devices, edge–cloud collaborative computing has emerged as a promising paradigm. By offloading computation-intensive tasks to the cloud while retaining latency-sensitive operations on the edge, this approach seeks to enable responsive and intelligent systems powered by LLMs.

While recent surveys have explored the deployment of Artificial Intelligence-Generated Content (AIGC) in mobile edge networks, integrated air-ground edge-cloud systems, and on-device large language models (LLMs) within mobile edge intelligence (MEI), these works primarily focus on specific application scenarios, system-level architectures, or isolated technical components. In contrast, this survey presents a comprehensive and systematic investigation into collaborative edge-cloud AI systems, with a particular emphasis on the cooperative inference and training of large and small language models.

1.2 Definition of Cloud-Edge Collaborative with Large and Small Language Models

Cloud–Edge Collaboration with Large and Small Language Models refers to an intelligent inference paradigm in which general-purpose large-scale models in the cloud cooperate with lightweight small models on edge devices. This approach aims to combine the strengths of both model types to enable efficient, personalized, and low-latency intelligent services.

- **Complementary Capabilities:** Cloud-based large language models possess powerful reasoning, multi-modal understanding, and generalization abilities, making them well-suited for handling complex tasks. In contrast, edge-deployed small models are optimized for real-time perception and response, offering high efficiency and flexible deployment.
- **Collaborative Inference:** By offloading certain inference or adaptation tasks to edge devices, cloud–edge systems can jointly perform reasoning and learning, thereby reducing service latency and enhancing the model's ability to deliver personalized responses.
- **Privacy and Efficiency:** Edge models can process raw user data locally, minimizing the need for data transmission to the cloud. This not only alleviates the computational load on centralized servers but also strengthens privacy protection and improves overall system efficiency.

1.3 Research Questions and Structure Overview of the Survey

This survey aims to systematically review the development status, key technical challenges, and future research directions of the collaborative inference paradigm based on large and small language models in edge–cloud environments. The discussion is structured around the following core research questions (RQs):

RQ1: What is the edge–cloud collaborative inference paradigm with large and small models? What are its fundamental concepts and system architecture? This question focuses on the foundational definition of the paradigm and the division of roles between large and small models. It emphasizes their complementary capabilities in terms of computational power, deployment locations, and task responsibilities.

RQ2: What are the key challenges in building efficient and secure edge–cloud collaborative inference systems? This includes core design issues such as task partitioning mechanisms, communication and latency

management, personalized adaptation, and privacy preservation. It investigates the inherent coupling difficulties in system-level coordination between edge and cloud components.

RQ3: What are the representative collaborative inference models and optimization strategies in existing research? This question explores lightweight small model design, prompt-based enhancement for large models, dynamic task scheduling mechanisms, and edge-aware model updates. It highlights the trade-offs between efficiency and performance across various strategies.

RQ4: What are the current limitations and future development trends in this field? It discusses potential directions and challenges, including multimodal integration, continual learning capabilities, cross-device collaboration, and trustworthy and secure AI.

To address the above research questions systematically, the structure of this survey is organized as follows:

- **Section 2** introduces the fundamentals of edge–cloud collaborative large/small language models, covering system architectures, edge device constraints, and trends in large model development, thereby addressing (**RQ1**).
- **Section 3** explores task allocation and inference strategies, with a particular focus on dynamic scheduling mechanisms and draft–refine collaborative verification frameworks. This section addresses how models on edge and cloud collaboratively execute inference tasks efficiently (**RQ2**).
- **Section 4** provides a comprehensive overview of multi-model collaborative architectures and lightweight deployment strategies, including role-specific model assignment, parameter-efficient fine-tuning, cross-domain knowledge transfer, and quantization/pruning in edge deployments, thus addressing (**RQ3**).
- **Section 5** summarizes benchmarking systems, evaluation metrics, and domain-specific deployment practices. It supports model comparison and practical implementation from a systems evaluation perspective.
- **Section 6** looks toward future developments, identifying key open challenges in generalizability, green AI, privacy-preserving mechanisms, and multi-agent collaboration across heterogeneous devices, thereby addressing (**RQ4**).

The overall structure and logical framework of this survey are illustrated in Fig.

1.4 Scope and Organization of the Survey

This survey provides an in-depth analysis of the current landscape of edge–cloud collaboration for LLMs. We begin by outlining the foundational principles and system architectures that support such collaboration. We then explore recent advancements, highlighting key research efforts that tackle the aforementioned challenges. Finally, we discuss emerging trends, open problems, and potential future directions. The remainder of this paper is organized as follows: Section 2 introduces the taxonomy of edge–cloud collaborative strategies, Section 3 reviews existing techniques for model partitioning and optimization, and Section 4 discusses security and privacy concerns. We conclude with insights and outlooks in Section 5.

1.5 Contributions of This Survey

This survey makes the following key contributions to the emerging field of edge–cloud collaborative large and small language models (LLMs):

- **Unified Perspective on Edge–Cloud Collaborative Intelligence:** We establish a structured conceptual framework that systematically characterizes the interplay between large-scale foundation models and resource-constrained small models under edge–cloud paradigms. This bridges fragmented research across systems, models, and task levels, providing a coherent understanding of collaborative AI deployments.
- **End-to-End Collaboration Beyond Inference:** Unlike prior surveys focused primarily on inference scheduling, this work highlights a broader pipeline involving collaborative training, fine-tuning, and

prompt optimization across the edge and cloud. We particularly emphasize data locality, privacy-preserving adaptation, and lightweight personalization within collaborative loops.

- **Taxonomy of Multi-Model Collaboration and Optimization:** We present a fine-grained taxonomy of multi-model cooperation strategies, including hierarchical model roles, adapter-based modularity, and parallel pipeline execution. Moreover, we explore lightweight deployment techniques such as parameter-efficient tuning, modular adapters, cross-domain knowledge distillation, and quantization in edge contexts.
- **Speculative Decoding and Collaborative Verification:** We survey emerging speculative and cooperative inference mechanisms (e.g., draft–refine frameworks and skeleton completion strategies), which offer new directions in reducing latency while preserving response quality in hybrid environments.
- **Benchmarks, Deployment Practices, and Open Challenges:** We collect and analyze real-world deployment cases and evaluation protocols tailored to collaborative LLM settings, and we identify open challenges such as generalization, privacy-preserving frameworks, agent orchestration, and green design—setting the stage for future research and system co-design.

1.6 Related Surveys and Their Scope

Unlike the survey [1], which centers on deploying AIGC services (e.g., ChatGPT, DALL·E) at the mobile edge to enable real-time, personalized content generation, our work is not confined to generative applications. Instead, we provide a broader design space encompassing model scheduling, inference collaboration, and learning paradigms for general-purpose language models. Compared to survey [2], which targets UAV-assisted inference in 6G environments and proposes task-specific optimization strategies under strict communication constraints, our survey abstracts from such application-specific contexts and formulates generalizable frameworks for task allocation and model coordination across diverse edge-cloud scenarios. Furthermore, while the third survey [3] focuses on deploying LLMs at the mobile edge to address latency and privacy concerns, our study goes a step further by investigating adaptive collaboration mechanisms between resource-constrained edge models and powerful cloud-based LLMs, particularly under the constraint of preserving the original pre-trained model structure. Overall, this survey aims to unify existing strategies into a cohesive taxonomy, covering system architecture, dynamic task scheduling, speculative decoding, multi-model collaboration, lightweight deployment, and security concerns. By identifying key design principles and open challenges, we provide a foundation for future research on building efficient, scalable, and privacy-aware edge-cloud collaborative systems for large-scale AI applications.

2 Fundamentals of Edge–Cloud Collaborative AI Systems

2.1 Edge–Cloud Computing Architecture

The edge-cloud collaborative inference framework strategically distributes inference tasks between large cloud-based models and small on-device models to optimize computational efficiency and reduce latency. Various approaches have been proposed to balance the trade-off between model performance, communication cost, and real-time responsiveness. HybridSD [4] focuses on visual semantic planning, where the cloud-based large model plays a dominant role in the early stages of the denoising process, establishing structural integrity and semantic clarity. The small model on the edge, while less effective in semantic integration, demonstrates its strength in later stages by enhancing perceptual consistency and refining details. This division of labor ensures both efficiency and high-quality visual generation. CE-CoLLM [5] introduces a confidence-based dynamic inference allocation strategy. High-confidence tokens are processed on the edge, while low-confidence tokens are offloaded to the cloud LLM. This approach significantly reduces communication overhead while maintaining inference accuracy. Additionally, an early exit mechanism is employed, allowing intermediate-layer termination to minimize

unnecessary computation, thereby improving system responsiveness. Compared to HybridSD and HybridRAG, CE-CoLLM specifically targets token-level efficiency in large language model inference.Enhanced hybrid inference [6] extends the paradigm by incorporating local optimization and adaptive task offloading, aiming to minimize latency and dependence on continuous internet connectivity. Unlike CE-CoLLM, which primarily focuses on token-wise decision-making, this approach dynamically adjusts model parameters based on user-specific data. By leveraging a utility function to quantify the personalized contribution of local and cloud computation, the system achieves an optimal balance between responsiveness and computational efficiency. Furthermore, an active inference approach based on expected free energy [7] offers a novel perspective on task offloading. By introducing a reward-free guiding mechanism and utilizing a generative model to predict environmental states, the approach optimizes inference paths without relying on large-scale reward models. Compared to traditional deep reinforcement learning (DRL), this method exhibits superior data efficiency and adaptability to dynamic workloads. Particularly in resource-constrained edge scenarios, it provides an optimal strategy for large-model inference offloading, ensuring both performance stability and computational efficiency.In summary, while all these methods share the common goal of optimizing edge-cloud inference by leveraging both local and cloud resources, their focuses and methodologies differ. HybridSD and HybridRAG emphasize task-specific optimization in vision and NLP, respectively, while CE-CoLLM and enhanced hybrid inference prioritize dynamic token- and task-level efficiency.

2.2 Characteristics and Constraints of Edge Devices

Edge devices are characterized by limited computational power, memory, and energy resources. These constraints significantly impact the feasibility of deploying large AI models, particularly large language models (LLMs), on-device. Additionally, heterogeneity in hardware and software platforms poses challenges for standardized model deployment. Ensuring real-time performance, robustness, and adaptability on edge devices requires lightweight model design, hardware-aware optimizations, and adaptive execution strategies tailored to dynamic edge environments.

2.3 Trends in Large and Small Language Models

Recent developments in language modeling reveal a divergence in model scaling strategies. While LLMs such as GPT and PaLM continue to scale up in terms of parameters and capabilities, there is growing interest in the development of small language models (SLMs) that retain high efficiency and competitive performance within constrained environments. Techniques like model compression, quantization, and knowledge distillation have enabled the creation of edge-friendly SLMs, facilitating on-device inference without sacrificing too much accuracy. Understanding this dual trend is essential for designing collaborative systems that intelligently balance workload between large-scale cloud models and edge-deployed lightweight counterparts.

3 Overview of Collaborative Learning and Inference Paradigms

Several paradigms have emerged to facilitate collaborative AI between edge and cloud. Federated learning allows distributed edge devices to collaboratively train models without sharing raw data, preserving privacy. Split computing partitions neural network inference across edge and cloud nodes to reduce latency and energy consumption. More recently, adaptive inference and early exit mechanisms have been explored to dynamically determine inference depth based on runtime conditions. These paradigms highlight the rich design space of collaborative AI and underline the importance of task-aware and resource-aware coordination strategies in modern systems.

3.1 Task Allocation and Inference Strategies

With the widespread deployment of large language models (LLMs) in intelligent edge scenarios, the tension between their high resource consumption and inference latency has become increasingly prominent. A critical challenge in edge-cloud collaboration lies in the efficient partitioning of tasks and allocation of resources between edge devices and cloud servers, aiming to balance inference performance, energy efficiency, and privacy preservation. Recent studies have explored this challenge from three key perspectives—request scheduling, task orchestration, and computation offloading—proposing a variety of collaborative strategies that significantly enhance the usability and responsiveness of LLMs in resource-constrained environments.

3.1.1 Resource- and Uncertainty-Aware Task Allocation for Cloud-Edge Inference. In complex environments, dynamically adjusting the inference path based on request characteristics and system states remains a key challenge for ensuring service quality. PerLLM [8] introduces a personalized scheduling framework that integrates an upper-confidence bound strategy with a constraint-satisfaction mechanism. Targeting various LLM service types, it optimizes service deployment and resource allocation across edge and cloud, effectively balancing processing latency and energy consumption. Similarly, EdgeLLM [9] proposes a value-density-first (VDF) scheduling algorithm that prioritizes tasks dynamically by evaluating their cost-effectiveness in real time. Combined with threshold-based preemption and batch processing strategies, it significantly improves scheduling efficiency on edge servers. Unlike conventional approaches, these methods emphasize multi-objective joint optimization and environment-aware adaptability, enhancing both responsiveness and robustness in real-world scenarios. It effectively addresses the limitations of traditional scheduling algorithms (e.g., FCFS, EDF) in handling the high resource demands and stochastic arrival patterns characteristic of LLM workloads.

The structural characteristics of tasks offer new opportunities for fine-grained collaborative scheduling. Division-of-Thoughts (DoT) [10] introduces a task decomposer and a dependency graph scheduler to break down complex user queries into parallelizable subtasks, enabling identification of critical paths and model granularity adaptation. This significantly improves both inference parallelism and accuracy. SuperMoE [11] and EdgeMoE [12] adopt expert module management strategies, dynamically selecting subnetwork paths during inference based on routing networks or historical statistical features. By reducing the number of activated modules, these approaches minimize I/O overhead and memory usage. Focusing on task structure and modular model design, such methods enable flexible and efficient scheduling in heterogeneous edge-cloud computing environments.

PerLLM [8] addresses the problem of resource allocation in edge–cloud collaborative environments. It formulates the goal of minimizing the overall energy consumption of LLM services—including transmission, inference, and idle energy costs—as a Combinatorial Multi-Armed Bandit (CMAB) problem. To solve it, the authors propose the Constraint Satisfaction Upper Confidence Bound (CS-UCB) algorithm, where each action corresponds to a specific service assignment strategy, represented as a "super arm" consisting of service–server mappings. Through iterative online selection and reward estimation updates, the algorithm progressively learns an optimal allocation policy. This approach exemplifies a task scheduling mechanism where decisions on whether to invoke large or small models are driven by task complexity and resource conditions.

In contrast, the method proposed in [13] emphasizes the reverse aggregation of knowledge from edge to cloud, enabling joint modeling. In this framework, multiple independently trained edge models are uploaded to the cloud, where a gating network coordinates them by establishing linear projection paths among models. This forms a bottom-up architecture known as the Bottom-up Aggregated Inference Model (BAIM). Joint training on cloud-hosted public datasets enhances the system's generalization capability and supports efficient multi-task and multi-modal learning. This design enables the cloud to selectively activate specialized edge models, effectively implementing a "large-model-to-small-model" coordination strategy.

Building further, FS-GEN [14] approaches the problem from a cognitive systems perspective by categorizing standalone SLMs as System 1 (fast and intuitive) and LLMs (or their combinations) as System 2 (slow and

deliberative). It proposes a collaboration mechanism that is triggered based on uncertainty during inference. Experimental results reveal that only a small fraction of collaborative interactions—typically under 20%—are necessary in most scenarios, and the interaction frequency follows a predictable scaling law with respect to model parameter ratios. Moreover, the study identifies key conditions under which System 2 intervention becomes critical, particularly in high-uncertainty situations. This uncertainty-aware coordination scheme enables selective activation of large models to support small models, achieving predictable and efficient performance improvements with minimal communication overhead.

The work [15] proposes an operator-level edge–cloud collaborative scheduling strategy, in contrast to traditional layer-wise partitioning methods. By modeling the relationship between input sequence length, inference stages (prefill/decode), and operator characteristics (e.g., latency and bandwidth bottlenecks), the authors design a greedy scheduling algorithm that dynamically assigns operators based on their computational intensity and memory access ratio. Under resource-constrained conditions, linear and attention operators are prioritized for offloading to the cloud to alleviate local processing pressure. The strategy incorporates a latency constraint, enabling efficient collaborative inference while keeping communication overhead within acceptable bounds.

3.1.2 Computation Offloading and System-Level Optimization. In resource-constrained terminal environments, offloading inference or training workloads to the cloud is essential for enhancing overall system efficiency. Liu et al. [16] propose a multi-objective offloading optimization strategy that combines partial parameter fine-tuning (PEFT) with a hierarchical execution architecture. Low-level model layers are deployed on local devices, while high-level tasks are offloaded to edge servers. By employing fractional programming and the Concave-Convex Procedure (CCCP), their approach effectively decomposes and solves the non-convex optimization problem, balancing energy consumption, latency, and system stability. He et al. [17] move beyond conventional deep reinforcement learning (DRL) paradigms by introducing a reward-free active inference mechanism. This method directly optimizes state-space awareness and action selection during task offloading, demonstrating superior generalization and stability. Collectively, these studies—spanning algorithmic design, system-level optimization, and strategic decision-making—advance the efficiency and reliability of edge-cloud collaboration for large-scale inference in complex environments.

3.1.3 Integrated Collaboration and System Organization. Beyond traditional task and resource scheduling strategies, recent research has begun to explore architectural restructuring and system-level organization for edge–cloud collaboration. For instance, BAIM [13] constructs a bottom-up architecture by aggregating multiple edge models into the cloud via a gating network. This design enables joint inference and knowledge sharing across multi-task and multi-modal scenarios, achieving a dynamic balance between personalization and generalization. Expanding on this idea, Pan et al. [18] address the heterogeneity and trustworthiness of edge nodes by introducing a node recruitment mechanism that incorporates data alignment evaluation and security filtering. Their approach enhances the robustness of the collaborative system while improving the quality of selected edge participants. These efforts underscore a shift toward system-level coordination and integration, providing a structural foundation for more scalable and resilient edge–cloud collaborative frameworks. Building on this direction, the AdaptLink framework [19] innovatively distributes large multimodal language model (MLLM) inference tasks across a cluster of heterogeneous edge devices. By exploiting parallel execution and resource diversity, AdaptLink accelerates inference while maintaining system efficiency. Moreover, it introduces a backup block preloading mechanism, wherein critical model sub-blocks are cached on idle devices. This design allows backup devices to seamlessly take over inference tasks in the event of primary device failure, thereby improving fault tolerance and system availability.

3.1.4 Threshold-Based Decision and Queuing Algorithms.

4 Collaborative Inference and Load Balancing

Efficient collaborative inference and load balancing strategies have been explored to enhance the performance and adaptability of large language models (LLMs) in distributed environments. PipeEdge [**?**] employs pipeline parallelism to distribute LLM shards across devices, leveraging an automatic partitioning scheduler to optimize throughput. PerLLM models edge-cloud collaboration as a Constrained Multi-Armed Bandit (CMAB) problem and incorporates an Upper Confidence Bound (UCB)-based optimization algorithm. This enables dynamic adaptation to varying conditions, optimizing LLM service scheduling while meeting latency constraints and minimizing energy consumption. Additionally, PerLLM integrates constraint satisfaction mechanisms to handle scenarios where edge servers have high computational latency but significantly lower transmission delays. By effectively modeling action and state spaces, PerLLM reduces search complexity and enhances scheduling efficiency. PETALS [**?**] aggregates idle computational resources from multiple research sources to ensure reliable inference on heterogeneous devices that may join or leave the network dynamically.

4.1 Computation and Storage Optimization

Efficient inference of large language models (LLMs) requires a balance between computational performance and storage overhead. Parameter offloading strategies improve computational efficiency in batch inference scenarios by storing part of the model parameters in external media and leveraging background pre-scheduling techniques. However, these methods face bottlenecks in interactive inference due to parameter loading latency. Additionally, the Layer-wise Unified Compression (LUC) method employs layer-wise quantization and pruning strategies to reduce model redundancy. By integrating adaptive layer adjustments and a voting mechanism, it minimizes storage overhead while maintaining inference flexibility and accuracy. At the hardware level, Edge-LLM incorporates a hardware scheduling module that dynamically adjusts quantization bit width, pruning sparsity, and segment update strategies, enabling efficient inference under irregular computational patterns. These studies provide effective solutions for optimizing end-cloud collaborative inference from both algorithmic and hardware perspectives, offering significant support for improving inference performance and resource utilization [20].

4.2 Communication Optimization in Collaborative Inference

Efficient communication is essential for enabling seamless collaboration between small-scale models at the edge and large-scale models in the cloud. One approach involves utilizing specialized AI models, such as GPT, to analyze user requests and decompose them into executable task sequences. By leveraging request prefixes, contextual information, and historical interactions, GPT intelligently selects a suitable combination of AI models to execute the tasks. To further reduce inference latency, mobile edge computing frameworks are employed to offload computational tasks from sensor devices to edge servers. A hierarchical deep neural network structure determines an optimal partitioning point, splitting the model into a lightweight device-side component and a more computationally intensive edge-side component. The intermediate features generated on the device are then transmitted to the edge server for further processing. Additionally, to minimize communication overhead, entropy models are introduced for feature compression, optimizing the trade-off between inference accuracy and transmission cost [21]. The EdgeShard framework extends this optimization by introducing a dynamic token-level edge-cloud collaboration mechanism tailored for large language models (LLMs). By deploying small-scale language models (SLMs), such as TinyLlama, on edge devices and facilitating token-level interactions with cloud-based LLMs, EdgeShard achieves low-cost inference while maintaining near-LLM quality. A core aspect of this approach is the identification of "critical tokens"—tokens that are crucial to inference accuracy but challenging for SLMs to generate independently. Statistical analysis of generated outputs reveals that SLMs and LLMs produce similar results for simple tokens, while significant discrepancies arise for more complex tokens. These advancements in communication optimization play a crucial role in enabling efficient and scalable

collaborative inference between edge and cloud systems, improving both response time and resource utilization in distributed AI environments.

4.3 Historical Enhancement and Retrieval-Augmented Collaborative Inference in Edge–Cloud Collaboration

Despite recent progress, on-device large language models (LLMs) still lag behind their powerful cloud-based counterparts in terms of performance. Interestingly, user queries to cloud APIs often resemble those directed at on-device models. Moreover, responses from cloud-based LLMs typically exhibit high quality and can serve as valuable reference answers. This observation motivates a new line of research: leveraging historical interactions between users and cloud LLMs to enhance on-device inference at minimal cost.

Driven by this insight, recent studies have explored approaches that combine historical data augmentation (e.g., past question–answer pairs or multimodal interaction records) with edge-side caching mechanisms, aiming to strike a better balance between inference quality and system efficiency. Ding et al. [22] propose building an external data store on mobile devices to retain users' historical interactions with cloud-based LLMs. By employing nearest-neighbor retrieval, the system enhances the reasoning capabilities of the on-device LLM. While maintaining a full data store significantly improves generation quality, it also introduces substantial inference latency. To address this, the authors introduce both offline and online subset selection algorithms—under storage constraints—with provable performance guarantees, enabling flexible and scalable deployment. At the system level, the VELO framework [23] extends historical enhancement to edge-side scheduling. By deploying a vector database on edge servers to cache embeddings of past LLM requests and responses, VELO computes similarity between incoming requests and cached entries to dynamically determine the optimal handling strategy: directly returning a cached answer, retrieving enhanced context before querying the cloud, or invoking the cloud LLM directly. This design significantly reduces cloud load and response latency for repeated or similar queries, offering an efficient edge–cloud collaborative LLM service. Targeting recommendation scenarios, LSC4Rec [24] introduces a cloud–edge collaborative recommendation framework that combines joint training and inference. It leverages the cloud-based LLM to generate diverse candidate items and enhances local lightweight ranking models (SRMs) on the edge. This hybrid approach facilitates real-time user preference modeling and candidate reranking on the device, while also minimizing training and communication overhead via intelligent request strategies. The synergy between the LLM's generalization capability and the SRM's fast responsiveness leads to both improved recommendation quality and system efficiency. In the multimodal setting, CDC-MMPG [25] extends the idea of historical enhancement to personalized model migration. The model incorporates a Fast Domain Adapter (FDA) that trains a global model using historical multimodal data and generates personalized parameters based on real-time device inputs. It further employs an Anchor Frame Distribution Reasoning (ADR) module to standardize modality inputs, facilitating downstream prediction by a lightweight on-device model. This architecture enables adaptive and efficient handling of diverse and dynamic multimodal inputs. HybridRAG [**?**] introduces an efficient framework that integrates cloud-based retrieval models, large language models (LLMs), and lightweight on-device predictors. The system operates asynchronously, where the cloud retrieves relevant documents and compresses them into memory units using the LLM. These memory units are then transmitted to the edge, where a small model refines its predictions based on this additional context. By distributing retrieval and generation tasks across the edge and cloud, HybridRAG effectively balances computational efficiency and predictive accuracy.

Although these works address different tasks and adopt varied system designs, they share a common foundation: the structured and selective use of historical interaction data to enhance local inference. Whether through retrieval-augmented generation, cache-based scheduling, or collaborative inference, these approaches reveal a growing trend—history-aware designs are reshaping the paradigm of edge–cloud LLM collaboration. Their technical distinctions—such as Ding et al. [22]'s focus on theoretical subset selection, VELO's [23] emphasis on

adaptive scheduling, LSC4Rec's [24] integration of ranking and generation, and CDC-MMPG's [25] personalized multimodal modeling—reflect the rich diversity and adaptability of this emerging research direction.

4.4 Speculative Decoding and Collaborative Verification

With the widespread adoption of large language models (LLMs) in intelligent terminal applications, a key challenge in collaborative edge–cloud inference is how to enhance inference efficiency and reduce latency without compromising generation quality. Conventional LLM inference is typically computationally intensive and latency-prone, making it unsuitable for resource-constrained edge devices. Speculative decoding, an emerging strategy for efficient generation, addresses this issue by allowing the edge model to produce an initial "draft" output, which is subsequently verified and refined by a more powerful cloud-based model. This approach offers a promising paradigm for low-latency, high-reliability collaborative generation. At its core, this mechanism is built on the principle of "lightweight drafting + precise verification," striking a balance between computational efficiency and generation quality. The edge model is responsible for rapidly generating high-probability candidate outputs, while the cloud model conducts semantic consistency checks, corrects uncertainties, and enriches the content with additional details—significantly reducing overall inference cost. Furthermore, token-level verification mechanisms—such as token tree search and structural consistency checks—further enhance the robustness and stability of collaborative inference. This chapter provides a comprehensive overview of speculative decoding architectures and techniques in edge–cloud collaborative scenarios. It covers key components including the draft–refine framework, draft complementation strategies, skeleton completion mechanisms, and token-level validation. Through the coordinated design of these techniques, edge devices and cloud-based LLMs can work in tandem to enable efficient, secure, and high-quality generation for pervasive intelligent applications.

To further optimize communication efficiency, the framework replaces full data transmissions with token ID exchanges, significantly reducing both data volume and transmission frequency per inference transaction [26].

4.4.1 Edge Draft and Cloud Validation. Most large language models (LLMs) employ an autoregressive, token-by-token generation mechanism, where each token is sequentially produced and used as input for predicting the next. While this approach yields high-quality outputs, it introduces considerable inference latency, which significantly hinders the deployment of LLMs in real-time or resource-constrained environments. To address this limitation, recent studies have proposed speculative decoding—a technique that introduces a lightweight drafting model to pre-generate multiple candidate tokens in parallel, thereby accelerating the decoding process. In this paradigm, the primary model (i.e., the base LLM) no longer generates tokens from scratch. Instead, it validates candidate sequences generated by the drafting model and decides whether to accept them based on certain criteria, such as probabilistic consistency. If the drafts are verified, they are adopted directly as the final output, significantly reducing decoding steps. Otherwise, the system reverts to the standard step-by-step generation procedure.

A representative early work in this area is BiLD [27], which systematically integrates size-based model collaboration and verification strategies to optimize decoding. BiLD comprises a small autoregressive model for initial text generation and a larger non-autoregressive model that assists when needed. Through a fallback mechanism, the small model delegates uncertain predictions to the large model, while a rollback strategy enables the correction of inaccurate outputs. Additionally, BiLD introduces prediction alignment techniques to maintain consistency between models at a low computational cost. Building on this foundation, RSD [28] and SpecExec [29] algorithm further enhance the collaboration between the lightweight drafting model and the more powerful target model by incorporating a controlled bias strategy that prioritizes high-reward outputs. Unlike conventional speculative decoding methods that emphasize strict unbiasedness, RSD introduces a procedural reward model to assess intermediate decoding steps and dynamically determine whether to invoke the target model, achieving a more effective trade-off between computational cost and output quality. Another innovative framework is SMART [30], which draws inspiration from the concept of cognitive scaffolding. SMART selectively enhances

small language model (SLM) reasoning through targeted guidance from a large language model. It uses score-based evaluations to identify uncertain reasoning steps and injects LLM-generated reasoning only when necessary. By formulating structured reasoning as an optimal policy search problem, SMART guides inference trajectories toward correct solutions without resorting to exhaustive sampling.

In interactive AI applications—such as intelligent question answering, voice assistants, and real-time translation systems—the Quality of Experience (QoE) is often highly dependent on the responsiveness and fluency of the system. Specifically, QoE is typically measured using two key latency metrics. The first is Time-To-First-Token (TTFT), which quantifies the delay between a user request and the generation of the first token, directly influencing the user's perception of system responsiveness. The second is Token-By-Token Time (TBT), which denotes the average time interval between consecutive tokens, reflecting the coherence and rhythm of the generation process. However, traditional autoregressive mechanisms, where each token depends on the preceding output, struggle to achieve both low TTFT and low TBT—particularly in large language models. To address this trade-off, DiSCo [31] proposes a device-server collaborative scheduler that adaptively routes requests and migrates the generation process between edge devices and servers. This approach aims to optimize QoE while maintaining cost efficiency. DiSCo introduces a cost-aware scheduling strategy that leverages the predictable inference speed of on-device models and the flexibility of server-based inference to dynamically manage task allocation. Building on this idea, AdaServe [32] further enhances QoE optimization under diverse service-level objectives (SLOs). It estimates the probability that each speculative token can be verified by the target model, using the logits from the drafting model. Based on this estimation, AdaServe dynamically determines the minimum number of tokens that must be accepted to satisfy specific TTFT and TBT requirements, thereby enabling efficient SLO-customized inference.

On another front, Fuzzy Speculative Decoding (FSD) [33] introduces the idea of allowing moderate deviations from the target distribution to significantly improve inference speed without severely compromising output quality. Across multiple benchmark evaluations, FSD achieves an improvement of approximately five tokens per second over standard speculative decoding, while incurring only a 2% drop in accuracy. In certain scenarios, FSD even matches the output quality of standard speculative decoding at a rate exceeding two additional tokens per second, indicating that strict distributional equivalence is not always necessary to preserve target model performance.DuoDecoding [34] takes advantage of hardware-level parallelism by deploying the drafting model on the CPU and the target model on the GPU. Simultaneously, it employs a hardware-aware budget optimization strategy that dynamically adjusts the draft generation length based on current hardware load. This approach effectively balances computational efficiency and output quality, maximizing the synergy of heterogeneous hardware resources to reduce wait time and overall inference latency.

4.4.2 Self-speculative decoding without auxiliary draft model training. However, some studies have pointed out that introducing an additional lightweight draft model can impose extra communication and storage overhead in practical deployment scenarios.

Conventional speculative decoding methods typically rely on separately trained lightweight language models to generate draft tokens, enabling the primary model to decode in larger strides. While this strategy improves inference speed, it also increases training costs and system complexity. To address this limitation, Kangaroo [35] proposes a training-free speculative decoding approach by reusing the shallow subnetwork and LM head of the target LLM to construct a self-draft model. This design significantly reduces training overhead and resource consumption. Moreover, traditional speculative decoding often employs a fixed drafting stride, which can lead to inefficiencies: time is wasted on simple tokens, while opportunities for speculative execution may be missed for more complex ones. Kangaroo introduces a confidence-based dynamic drafting mechanism that adaptively adjusts the drafting stride according to the confidence level of the self-draft model, thereby enhancing inference efficiency while maintaining output quality. EESD [36] introduces an early-exit architecture constructed from the first N layers of the network, enabling the generation of high-quality draft tokens through an additional trainable

single-layer Transformer module. To further enhance prediction accuracy, a self-distillation approach is employed. This design effectively balances the trade-off between generation speed and output quality while reducing overall computational cost. Building on this idea, SWIFT [37] proposes a novel self-speculative decoding paradigm that eliminates the need for external draft models by exploiting the inherent layer sparsity in large language models (LLMs). Its core design consists of two key components: (1) a context-aware dynamic layer optimization strategy that selects skip-layer configurations adaptively during generation, using a combination of Bayesian optimization and random search to construct task-specific draft pathways; and (2) a confidence-aware drafting mechanism that dynamically controls the length of draft generation based on token-level confidence, enhanced with multi-candidate verification to ensure output accuracy and stability. This method operates entirely within the original model structure and, without any additional training, demonstrates the first self-speculative decoding framework based on layer skipping, achieving a balance between decoding efficiency and output consistency.

4.4.3 Semantic and Skeleton Completion (Edge Skeleton + Cloud Elaboration). The draft-refinement paradigm divides the text generation process into two complementary stages: edge-side drafting and cloud-based refinement. Initially, a lightweight edge model generates a coarse-grained draft based on local inputs and contextual information, serving as a low-cost approximation of the target output. This enables fast, offline, and responsive inference. Subsequently, the cloud performs selective refinement on the draft, without regenerating the entire sequence, to enhance output quality. This design reduces cloud-side computation and latency, while preserving user privacy and improving the overall interaction experience. Recent studies [26] have demonstrated that edge-based small language models (SLMs) are capable of handling simple queries and generating the structural backbone of responses. Even minimal corrections made by large language models (LLMs) can significantly enhance the final output. For instance, supplying a single correct token from the LLM can resolve over 120 cases; replacing just three tokens rectifies approximately 75% of test examples. In this framework, the SLM produces an extensible initial output, while the refinement stage selectively replaces critical tokens based on LLM outputs. The corrected positions are then used by the SLM to regenerate the subsequent sequence. The combination yielding the best performance is retained as the upper-bound reference for hybrid decoding.

This paradigm is particularly well-suited for latency-sensitive applications such as mobile assistants, augmented reality systems, and dialogue agents, where both responsiveness and output quality are crucial. To further improve throughput and reduce inference latency, PICE [38] proposes a progressive inference scheme, in which the LLM generates a task sketch on the cloud, and the SLM concurrently fills in the details on the edge. In privacy-sensitive environments, achieving effective collaboration without compromising user data remains a key research challenge. CoGenesis [39] introduces two collaborative paradigms: (1) the sketch-based approach, where the LLM provides a high-level task plan and the SLM fills in specific details—thereby avoiding exposure to raw private data; and (2) the logit-based approach, where the LLM shares output probabilities (logits) to guide SLM generation, enabling knowledge transfer without direct data sharing.

4.4.4 Token-Level Tree Validation. In speculative decoding, the verification mechanism plays a critical role in ensuring output quality. However, the verification process often incurs non-negligible computational overhead, especially on resource-constrained devices, where a single verification can take several seconds. Thus, determining *when* to trigger verification becomes a key factor in overall efficiency: verifying too early may waste computation if no errors have occurred yet, while verifying too late risks accumulating uncorrectable errors, rendering the entire draft invalid and wasting all prior computation. Despite this, many existing approaches still rely on simple heuristics such as fixed token counts or stride lengths to determine verification timing, which fail to adapt optimally across varying model architectures and dynamic contexts.

Moreover, in cascaded LLM frameworks, I/O and computational resources are often highly asymmetric: blocking execution of the main model, frequent context switches, and idle time while waiting for validation can significantly hinder end-to-end throughput. Against this backdrop, constructing a *token tree* emerges as a promising alternative

for improving inference efficiency. Unlike linear decoding, token trees allow each node to branch into multiple candidate paths, enabling broader exploration of the output space and reducing wasted computation from single-path errors. Along this line, LLMCad [40] departs from the conventional paradigm of linear sequence generation with per-token verification and instead adopts a tree-based generation and validation framework. Its core techniques include: (1) a confidence-based branching scheduler (pacer) that regulates the expansion of candidate paths to avoid overcommitting resources to erroneous branches, and (2) a tree decoder that expands all branches in parallel without frequent context switching. Additionally, LLMCad introduces a non-autoregressive token tree verifier that simultaneously validates and corrects all branches in a single iteration. This approach significantly reduces verification frequency and switching overhead, while improving error detection efficiency and hardware parallelism. Building upon this, AdaServe [32] and EdgeLLM [36] further optimize token tree construction by maximizing the number of tokens verified per invocation of the main model. Specifically, it builds a candidate token tree upon each request and dynamically selects the optimal token branches to maximize system throughput under given service-level objectives (SLOs). To bridge the gap between short-context training and long-context inference, **LongSpec** [41] proposes a novel positional indexing mechanism that enables seamless transition from short to long contexts. It also introduces an attention aggregation mechanism that combines fast-path prefix computation with standard attention supporting tree-structured masks, achieving both latency and memory efficiency under token tree decoding. Experimental results demonstrate that LongSpec delivers strong performance across a range of long-context tasks—including warehouse-scale code completion, long-form summarization, and O1-style complex reasoning—while substantially reducing inference latency. Finally, **OPT-Tree** [42] addresses a key limitation of prior speculative decoding methods that rely on fixed heuristic tree structures, which may fail to maximize accepted token length during verification. OPT-Tree introduces a dynamically adjustable tree construction algorithm that builds an expectation-optimal candidate tree at each decoding step using a greedy strategy. By leveraging a probabilistic prediction model to prune branch nodes dynamically, it significantly increases the number of valid tokens generated per step under a fixed node budget.

However, the above methods have not specifically addressed the issue of pipeline parallelism between the draft model and the target model. In fact, during the verification phase of the target model, the draft model can continue generating candidate tokens, which helps improve inference efficiency. To leverage this potential, SLiM [43] introduces an additional hypothesis selection phase between the traditional speculation and verification stages, forming a three-stage inference process: speculation, selection, and verification. In the speculation phase, SLiM employs multiple prediction heads to generate diverse candidate sequences in parallel, thereby increasing coverage of the generation space. During the selection phase, it incorporates a lightweight posterior estimation module that constructs a dynamic sparse binary correlation matrix to refine the probability of each candidate sequence. This mechanism effectively filters out low-confidence hypotheses and retains high-quality candidates for the final verification step.

5 Collaborative Architectures and Model Optimization

5.1 Multi-Model Collaborative Architectures

5.1.1 Hierarchical Model Cooperation (Backbone + Adapter).

5.1.2 Role-Specific Model Assignment (Summarizer, Answerer).

5.2 Lightweight Deployment and Knowledge Transfer

5.2.1 Parameter-Efficient Fine-Tuning (LoRA, BitFit).

5.2.2 Adapter-Based Modular Designs.

5.2.3 Cross-Domain Knowledge Transfer and Distillation. Despite the advantages of small language models (SLMs) on edge devices—such as low latency and improved privacy—their capabilities remain inferior to those of powerful cloud-based large language models (LLMs). To enhance the inference performance of SLMs in complex tasks, recent research has increasingly focused on mechanisms such as weight transfer, reinforcement learning, and knowledge distillation. These approaches aim to facilitate collaborative edge-cloud training that enables efficient knowledge transfer from the cloud to the edge, all while minimizing communication and computational overhead.

GKT [44] introduces a lightweight guidance-based cooperation mechanism between large and small models, wherein the cloud-side LLM (teacher) generates task-specific, compact prompts based on user input. These prompts guide the small model (student) deployed at the edge to perform inference. Without relying on model fine-tuning, vocabulary alignment, knowledge distillation, or speculative decoding, GKT offers a communication-efficient, customizable, and latency-friendly solution suitable for deployment in resource-constrained environments. DC-CCL [45] proposes a vertical model partitioning strategy that separates the base model into a large sub-model deployed in the cloud and a small sub-model executed on the device. While the large sub-model is responsible for learning from cloud-hosted data, the small sub-model handles local inference. To enable the small sub-model's optimization, which depends on gradients derived from the cloud model's outputs, the authors introduce a lightweight proxy model trained via knowledge distillation. This proxy model, deployable on edge devices, approximates the behavior of the cloud-side model, thereby reducing communication costs while maintaining effective training guidance. FedMKT [46] explores bidirectional knowledge transfer between cloud-based LLMs and client-side SLMs. On one hand, it adapts LLM capabilities to enhance SLMs; on the other, it incorporates client-specific domain knowledge into the server model. By leveraging Minimum Edit Distance (MinED) for token-level alignment, the framework enables selective and semantically aligned knowledge transfer, mitigating the challenges of heterogeneity and improving mutual performance. Extending this line of work to multimodal scenarios, CD-CCA [47] introduces a dynamic edge-cloud update mechanism. The framework uses a utility-based sample selection (UTS) strategy to upload high-value samples from the device to the cloud. The cloud then refines the global model using adaptive knowledge distillation (AKD) and compresses the resulting updates via a dynamic weight compression (DWC) technique. These lightweight updates are subsequently returned to the device, enabling near-lossless model enhancement with minimal communication cost. Unlike traditional unidirectional distillation or static updates, these methods emphasize structural, compressed, and adaptive knowledge transfer strategies. DC-CCL focuses on gradient guidance via proxy models, FedMKT constructs bidirectional transfer pipelines grounded in semantic alignment, and CD-CCA centers on multimodal update compression under bandwidth constraints. Meanwhile, Pan et al. [18] propose a high-quality edge node selection mechanism to support reliable collaborative training. These complementary approaches collectively enrich the design space of edge-cloud model adaptation and offer practical pathways for deployment across diverse application scenarios.

5.2.4 Quantization and Pruning in Edge Deployments.

5.3 Secure Multi-Party Computation and Homomorphic Encryption

5.4 Differential Privacy for Language Models

6 Benchmarks, Datasets, and Evaluation Protocols

6.1 Benchmark Systems and Metrics, Datasets

6.2 Evaluation of Trade-offs (Latency, Accuracy, Cost, Privacy)

6.3 Application of terminal cloud size model in vertical field

As intelligent systems increasingly demand both low-latency response and complex reasoning capabilities, edge–cloud collaborative architectures integrating large and small language models have demonstrated promising potential across various vertical domains.

In the domain of autonomous driving, ADAS [48] proposes a fine-grained task partitioning framework between edge and cloud. Specifically, a lightweight multimodal model (CogVLM2) is deployed at the edge to handle latency-sensitive tasks such as object detection, lane keeping, and emergency obstacle avoidance. Meanwhile, a large model (ChatGPT-4o) resides in the cloud to perform high-level reasoning, including route planning, global decision-making, and inter-vehicle coordination. This design addresses the limitations of rule-based or single-model ADAS systems, which often lack common-sense understanding and the ability to fuse multisensory data, thus significantly enhancing adaptability and safety in dynamic traffic scenarios. Building on this direction, EC-Drive [49] introduces an on-demand large-model activation mechanism. A lightweight LLM is deployed at the edge to handle most routine driving tasks with minimal latency. The cloud-based large model (e.g., GPT-4) is only engaged when "data drift" is detected—such as new obstacles or changing traffic patterns. This event-driven scheduling strategy enhances both the robustness and resource efficiency of the system in uncertain environments. While these approaches differ in system design and application focus, they collectively reflect a common architectural paradigm: edge devices are responsible for real-time responsiveness, while the cloud provides high-capacity reasoning capabilities. Departing from the traditional monolithic inference model, these systems leverage dynamic task awareness and role-specific model collaboration. Notably, ADAS emphasizes long-term multi-task coordination between edge and cloud, whereas EC-Drive adopts a conditional reasoning strategy that activates resources based on situational changes. Together, these efforts signal a broader trend in vertical applications toward structural elasticity, model heterogeneity, and intelligent task orchestration in edge–cloud collaborative systems.

7 Open Challenges and Future Directions

7.1 Edge–Cloud Synergy for Agent Intervention

7.2 Green AI and Energy-Efficient Design

7.3 Privacy-Preserving and Secure Collaboration Mechanisms

As large language models (LLMs) are increasingly deployed in edge–cloud collaborative scenarios, the tension between the sensitivity of user data and the adaptability of models has become a critical challenge. Traditional approaches tend to focus on improving inference efficiency, often overlooking privacy risks and security issues inherent in training and updating models. Consequently, achieving efficient collaborative optimization while preserving data locality has emerged as a key research direction.

Liu et al. [50] proposed an end-to-end training loop architecture that integrates edge proxies with lightweight adapters. In this framework, resource-intensive training tasks are offloaded to the cloud, while edge devices handle sensitive data processing and prompt optimization. This approach overcomes the limitations of existing frameworks that support only inference-level collaboration, enabling joint enhancement of data privacy (through data localization) and model adaptability (via cloud-assisted optimization). It is particularly effective in addressing

the challenges of prompt tuning for black-box LLMs in edge environments. From the perspective of inference collaboration, SuperICL [51] introduced a plugin-based framework that integrates fine-tuned task-specific small models into the context of LLMs. By combining confidence labels with original inputs, it constructs enhanced prompts that allow the LLM to dynamically decide when to rely on the plugin's localized knowledge and when to fall back on its own generalized reasoning. This enables improved accuracy and cooperation efficiency without exposing raw task data.

While differing in system architecture and optimization granularity, both approaches embody a shift toward privacy-aware edge–cloud collaboration. Rather than relying on a traditional master–slave inference model, they emphasize task allocation and model interaction within trusted execution boundaries. These designs offer promising directions for secure and efficient reasoning in complex, privacy-sensitive environments.

DCPR [52] addresses privacy concerns during model migration by innovatively employing a diffusion model as the core architecture and implementing a three-stage collaborative mechanism for personalized recommendation. Specifically, the cloud leverages privacy-preserving global category sequences to train a diffusion model that captures general mobility patterns; the edge layer incorporates regional POI data to enhance geo-semantic features; and the device layer adapts to user-specific preferences via a lightweight patch-based fine-tuning module. A hierarchical and progressive knowledge transfer strategy is further introduced to significantly reduce on-device computational overhead while preserving user privacy.

Pan et al. [18] address the challenges of limited resources and privacy risks in deploying large-scale model services within cloud-edge collaborative frameworks by proposing a comprehensive solution. The core of their approach lies in a hybrid mechanism that enables the generation of lightweight proxy models in the cloud and privacy-preserving inference execution at the edge. On the cloud side, a proxy model is constructed using remote fine-tuning and transfer learning techniques, incorporating lightweight adapters and compression modules to mitigate the communication and security issues associated with transferring full-scale foundation models. On the edge side, a binary sensitive information detection model is employed to identify privacy-critical data. Detected sensitive elements are then obfuscated using symbolic masking strategies. Furthermore, conditional generative adversarial networks (cGANs) are utilized to synthesize substitute data that preserves key information while enhancing data privacy. In addition, the framework introduces a lightweight reciprocal knowledge transfer method and an optimal edge node recruitment strategy to improve generalization and robustness under highly heterogeneous edge environments, thereby reinforcing privacy protection across the system.

8 Conclusion

8.1 Main Findings of the Survey

8.2 Recommendations for Future Research

8.3 Potential Impacts and Prospects of AI Models on Devices

References

[1] Minrui Xu, Hongyang Du, Dusit Niyato, Jiawen Kang, Zehui Xiong, Shiwen Mao, Zhu Han, Abbas Jamalipour, Dong In Kim, Xuemin Shen, et al. Unleashing the power of edge-cloud generative ai in mobile networks: A survey of aigc services. *IEEE Communications Surveys & Tutorials*, 26(2):1127–1170, 2024.

[2] Shuhang Zhang, Qingyu Liu, Ke Chen, Boya Di, Hongliang Zhang, Wenhan Yang, Dusit Niyato, Zhu Han, and H Vincent Poor. Large models for aerial edges: An edge-cloud model evolution and communication paradigm. *IEEE Journal on Selected Areas in Communications*, 2024.

[3] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. Mobile edge intelligence for large language models: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 2025.

[4] Chenqian Yan, Songwei Liu, Hongjian Liu, Xurui Peng, Xiaojian Wang, Fangmin Chen, Lean Fu, and Xing Mei. Hybrid sd: Edge-cloud collaborative inference for stable diffusion models. *arXiv preprint arXiv:2408.06646*, 2024.

[5] Hongpeng Jin and Yanzhao Wu. Ce-collm: Efficient and adaptive large language models through cloud-edge collaboration. *arXiv preprint arXiv:2411.02829*, 2024.

[6] Teresa Peng, Liam Liu, Maya Gupta, Kabir Mehta, Aditya Nair, Saanvi Desai, and Priya Singh. Enhanced hybrid inference techniques for scalable on-device llm personalization and cloud integration. *contexts*, 8(17):18–25, 2024.

[7] Jingcheng Fang, Ying He, F Richard Yu, Jianqiang Li, and Victor C Leung. Large language models (llms) inference offloading and resource allocation in cloud-edge networks: An active inference approach. In *2023 IEEE 98th Vehicular Technology Conference (VTC2023-Fall)*, pages 1–5. IEEE, 2023.

[8] Zheming Yang, Yuanhao Yang, Chang Zhao, Qi Guo, Wenkai He, and Wen Ji. Perllm: Personalized inference scheduling with edge-cloud collaboration for diverse llm services. *arXiv preprint arXiv:2405.14636*, 2024.

[9] Fenglong Cai, Dong Yuan, Zhe Yang, and Lizhen Cui. Edge-llm: A collaborative framework for large language model serving in edge computing. In *2024 IEEE International Conference on Web Services (ICWS)*, pages 799–809. IEEE, 2024.

[10] Chenyang Shao, Xinyuan Hu, Yutang Lin, and Fengli Xu. Division-of-thoughts: Harnessing hybrid language model synergy for efficient on-device agents. In *Proceedings of the ACM on Web Conference 2025*, pages 1822–1833, 2025.

[11] Lyudong Jin, Yanning Zhang, Yanhan Li, Shurong Wang, Howard H Yang, Jian Wu, and Meng Zhang. Moe-G2: Optimizing collaborative inference for edge large language models. *arXiv preprint arXiv:2501.09410*, 2025.

[12] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. Edgemoe: Fast on-device inference of moe-based large language models. *arXiv preprint arXiv:2308.14352*, 2023.

[13] Yuqing Tian, Zhaoyang Zhang, Yuzhi Yang, Zirui Chen, Zhaohui Yang, Richeng Jin, Tony Q. S. Quek, and Kai-Kit Wong. An edge-cloud collaboration framework for generative ai service provision with synergetic big cloud model and small edge models. *IEEE Network*, 38(5):37–46, 2024.

[14] Kaiyan Zhang, Jianyu Wang, Ning Ding, Biqing Qi, Ermo Hua, Xingtai Lv, and Bowen Zhou. Fast and slow generating: An empirical study on large and small language models collaborative decoding. *arXiv preprint arXiv:2406.12295*, 2024.

[15] Fan Yang, Zehao Wang, Haoyu Zhang, Zhenhua Zhu, Xinhao Yang, Guohao Dai, and Yu Wang. Efficient deployment of large language model across cloud-device systems. In *2024 IEEE 37th International System-on-Chip Conference (SOCC)*, pages 1–6. IEEE, 2024.

[16] Chang Liu and Jun Zhao. Resource allocation for stable llm training in mobile edge computing. In *Proceedings of the Twenty-fifth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, pages 81–90, 2024.

[17] Ying He, Jingcheng Fang, F Richard Yu, and Victor C Leung. Large language models (llms) inference offloading and resource allocation in cloud-edge computing: An active inference approach. *IEEE Transactions on Mobile Computing*, 2024.

[18] Yanghe Pan, Zhou Su, Yuntao Wang, Shaolong Guo, Han Liu, Ruidong Li, and Yuan Wu. Cloud-edge collaborative large model services: Challenges and solutions. *IEEE Network*, 2024.

[19] Xinyi Hu, Zihan Chen, Kun Guo, Meng Zhang, and Howard Hao Yang. Adaptlink: A heterogeneity-aware adaptive framework for distributed mllm inference. In *AAAI 2025 Workshop on Artificial Intelligence for Wireless Communications and Networking (AI4WCN)*, 2025.

[20] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices. *arXiv preprint arXiv:2406.02532*, 2024.

[21] Yifei Shen, Jiawei Shao, Xinjie Zhang, Zehong Lin, Hao Pan, Dongsheng Li, Jun Zhang, and Khaled B Letaief. Large language models empowered autonomous edge ai for connected intelligence. *IEEE Communications Magazine*, 2024.

[22] Yucheng Ding, Chaoyue Niu, Fan Wu, Shaojie Tang, Chengfei Lyu, and Guihai Chen. Enhancing on-device llm inference with historical cloud-based llm interactions. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 597–608, 2024.

[23] Zhi Yao, Zhiqing Tang, Jiong Lou, Ping Shen, and Weijia Jia. Velo: A vector database-assisted cloud-edge collaborative llm qos optimization framework. In *2024 IEEE International Conference on Web Services (ICWS)*, pages 865–876. IEEE, 2024.

[24] Zheqi Lv, Tianyu Zhan, Wenjie Wang, Xinyu Lin, Shengyu Zhang, Wenqiao Zhang, Jiwei Li, Kun Kuang, and Fei Wu. Collaboration of large language models and small recommendation models for device-cloud recommendation. *arXiv preprint arXiv:2501.05647*, 2025.

[25] Wei Ji, Li Li, Zheqi Lv, Wenqiao Zhang, Mengze Li, Zhen Wan, Wenqiang Lei, and Roger Zimmermann. Backpropagation-free multi-modal on-device model adaptation via cloud-device collaboration. *ACM Trans. Multimedia Comput. Commun. Appl.*, 2025.

[26] Zixu Hao, Huiqiang Jiang, Shiqi Jiang, Ju Ren, and Ting Cao. Hybrid slm and llm for edge-cloud collaborative inference. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*, pages 36–41, 2024.

[27] Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W Mahoney, Amir Gholami, and Kurt Keutzer. Speculative decoding with big little decoder. *Advances in Neural Information Processing Systems*, 36:39236–39256, 2023.

[28] Baohao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324*, 2025.

[29] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[30] Yujin Kim, Euiin Yi, Minu Kim, Se-Young Yun, and Taehyeon Kim. Guiding reasoning in small language models with llm assistance. *arXiv preprint arXiv:2504.09923*, 2025.
[31] Ting Sun, Penghan Wang, and Fan Lai. Disco: Device-server collaborative llm-based text streaming services. *arXiv preprint arXiv:2502.11417*, 2025.
[32] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, et al. Adaserve: Slo-customized llm serving with fine-grained speculative decoding. *arXiv preprint arXiv:2501.12162*, 2025.
[33] Maximilian Holsman, Yukun Huang, and Bhuwan Dhingra. Fuzzy speculative decoding for a tunable accuracy-runtime tradeoff. *arXiv preprint arXiv:2502.20704*, 2025.
[34] Kai Lv, Honglin Guo, Qipeng Guo, and Xipeng Qiu. Duodecoding: Hardware-aware heterogeneous speculative decoding with dynamic multi-sequence drafting. *arXiv preprint arXiv:2503.00784*, 2025.
[35] Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. In *Advances in Neural Information Processing Systems*, 2024.
[36] Jiahao Liu, Qifan Wang, Jingang Wang, and Xunliang Cai. Speculative decoding via early-exiting for faster llm inference with thompson sampling control mechanism. *arXiv preprint arXiv:2406.03853*, 2024.
[37] Heming Xia, Yongqi Li, Jun Zhang, Cunxiao Du, and Wenjie Li. Swift: On-the-fly self-speculative decoding for llm inference acceleration. *arXiv preprint arXiv:2410.06916*, 2024.
[38] Huiyou Zhan, Xuan Zhang, Haisheng Tan, Han Tian, Dongping Yong, Junyang Zhang, and Xiang-Yang Li. Pice: A semantic-driven progressive inference system for llm serving in cloud-edge networks. *arXiv preprint arXiv:2501.09367*, 2025.
[39] Kaiyan Zhang, Jianyu Wang, Remo Hua, Biqing Qi, Ning Ding, and Bowen Zhou. CoGenesis: A framework collaborating large and small language models for secure context-aware instruction following. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4295–4312, 2024.
[40] Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. Llmcad: Fast and scalable on-device large language model inference. *arXiv preprint arXiv:2309.04255*, 2023.
[41] Penghui Yang, Cunxiao Du, Fengzhuo Zhang, Haonan Wang, Tianyu Pang, Chao Du, and Bo An. Longspec: Long-context speculative decoding with efficient drafting and verification. *arXiv preprint arXiv:2502.17421*, 2025.
[42] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. Opt-tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025.
[43] Chi-Heng Lin, Shikhar Tuli, James Smith, Yen-Chang Hsu, Yilin Shen, and Hongxia Jin. Slim: Speculative decoding with hypothesis reduction. In *Findings of the Association for Computa100tional Linguistics: NAACL 2024*, pages 1005–1017, 2024.
[44] Yao Yao, Zuchao Li, and Hai Zhao. Gkt: A novel guidance-based knowledge transfer framework for efficient cloud-edge collaboration llm deployment. *arXiv preprint arXiv:2405.19635*, 2024.
[45] Yucheng Ding, Chaoyue Niu, Fan Wu, Shaojie Tang, Chengfei Lyu, and Guihai Chen. Dc-ccl: Device-cloud collaborative controlled learning for large vision models. *arXiv preprint arXiv:2303.10361*, 2023.
[46] Tao Fan, Guoqiang Ma, Yan Kang, Hanlin Gu, Yuanfeng Song, Lixin Fan, Kai Chen, and Qiang Yang. FedMKT: Federated mutual knowledge transfer for large and small language models. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 243–255, 2025.
[47] Guanqun Wang, Jiaming Liu, Chenxuan Li, Yuan Zhang, Junpeng Ma, Xinyu Wei, Kevin Zhang, Maurice Chong, Renrui Zhang, Yijiang Liu, et al. Cloud-device collaborative learning for multimodal large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12646–12655, 2024.
[48] Yaqi Hu, Dongdong Ye, Jiawen Kang, Maoqiang Wu, and Rong Yu. A cloud-edge collaborative architecture for multimodal llms-based advanced driver assistance systems in iot networks. *IEEE Internet of Things Journal*, 2024.
[49] Jiao Chen, Suyan Dai, Fangfang Chen, Zuohong Lv, Jianhua Tang, and Laiquan Han. Edge-cloud collaborative motion planning for autonomous driving with large language models. In *2024 IEEE 24th International Conference on Communication Technology (ICCT)*, pages 185–190. IEEE, 2024.
[50] Ya Liu, Kai Yang, Yu Zhu, Keying Yang, Chengtao Jian, Wuguang Ni, Xiaozhou Ye, and Ye Ouyang. Grey-box prompt optimization and fine-tuning for cloud-edge llm agents.
[51] Canwen Xu, Yichong Xu, Shuohang Wang, Yang Liu, Chenguang Zhu, and Julian McAuley. Small models are valuable plug-ins for large language models. *arXiv preprint arXiv:2305.08848*, 2023.
[52] Jing Long, Guanhua Ye, Tong Chen, Yang Wang, Meng Wang, and Hongzhi Yin. Diffusion-based cloud-edge-device collaborative learning for next poi recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2026–2036, 2024.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009